

AI-Powered GitHub Repository Modernization Platform

Comprehensive Multi-Phase Development Master Plan

Technical Specifications · Architecture · Schemas · Timelines · Effort Estimates

Local-first · Zero external LLM API dependency · Ollama-powered

Project Goal	Audit, modernize, document, secure, and continuously improve GitHub repositories
Core Stack	Node.js, TypeScript, GitHub API, Ollama (local LLM)
Total Phases	16 (Phase 0 – Phase 15)
Estimated Total Duration	34 – 46 weeks (solo developer, part-time pace)
Document Version	2.1 - Refined Master Plan with version-pinned stack

Table of Contents

Executive Summary

This document expands the original repository modernization roadmap into a fully specified engineering master plan. Each phase now includes detailed sub-tasks, technical specifications, data schemas, API contracts, file/folder layouts, effort estimates, sequencing dependencies, risks, and concrete acceptance criteria. The system is designed to run entirely on local infrastructure: a self-hosted Ollama instance serves all LLM inference (using Qwen2.5-Coder or DeepSeek-Coder models), eliminating any dependency on OpenAI, Anthropic, or other paid API keys for the AI components. GitHub's REST and GraphQL APIs are used for all repository interaction.

The platform is structured as a TypeScript monorepo using a packages/apps split (Turborepo or Nx), allowing the rule engine, stack detector, LLM service, and specialized agents to be developed, tested, and versioned independently while sharing common types and utilities.

How to Read This Plan

- Each phase opens with a duration estimate and dependency list, so phases can be parallelized where safe.
- Sub-tasks are broken into Backend, Data, and AI/Logic tracks where applicable.
- Technical specification tables define exact schemas, endpoints, and config shapes — these are meant to be implementation-ready, not illustrative.
- Acceptance criteria at the end of each phase define "done" — use these as sprint exit checklists.

Overall Timeline Summary

Phase	Name	Duration	Depends On
Phase 0	Research & Architecture	1 week	None
Phase 1	Repository Discovery	1.5 weeks	Phase 0
Phase 2	Deterministic Repository Auditor	2 weeks	Phase 1
Phase 3	Stack Detection Engine	2 weeks	Phase 1, 2
Phase 4	Rule Engine	2.5 weeks	Phase 3
Phase 5	Local LLM Integration	2.5 weeks	Phase 0, 3
Phase 6	Modernization Planner	2 weeks	Phase 4, 5
Phase 7	Cleanup Engine	2 weeks	Phase 6
Phase 8	Security Agent	2 weeks	Phase 2, 3
Phase 9	Dependency Intelligence	2 weeks	Phase 3
Phase 10	Portfolio Optimizer	1.5 weeks	Phase 5, 6
Phase 11	Multi-Agent Architecture	3 weeks	Phase 5–10
Phase 12	Pull Request Automation	2.5 weeks	Phase 7, 11
Phase 13	Historical Analytics	2 weeks	Phase 2, 12
Phase 14	Continuous Monitoring	2 weeks	Phase 12, 13
Phase 15	Dissertation / Research Extension	4–6 weeks	All prior phases

Total estimated duration: 34–46 weeks at a sustained part-time pace (10–15 hrs/week), or roughly 16–20 weeks full-time. Phases 8 and 9 can run in parallel with Phases 4–6 since they only depend on the stack detector, not the rule engine.

Phase 0 — Research & Architecture

Estimated Duration	1 week	Dependencies	None
--------------------	--------	--------------	------

Establishes the technical foundation: scope, stack choices, local AI runtime, and the modular monorepo skeleton everything else builds on.

0.1 Define Project Goals and Scope

- Document the problem statement: developers accumulate repositories that drift out of date, lack documentation, and carry security risk.
- Define success metrics: % of repos with passing health score >80, average time-to-fix per finding, number of auto-generated PRs merged.
- Define non-goals explicitly: no support for private package registries in v1; no support for non-GitHub hosts (GitLab/Bitbucket) until Phase 15.
- Output artifact: `docs/scope.md` containing goals, non-goals, and success metrics.

0.2 Choose Node.js + TypeScript

- Node.js 24.x LTS recommended, Node.js 22.x LTS supported as the minimum runtime; TypeScript 6.0.x in strict mode.
- Package manager: pnpm (preferred for monorepo workspace efficiency) over npm/yarn.
- Monorepo tooling: Turborepo for build orchestration and caching; alternative: Nx if richer code-gen/dependency graph visualization is desired.
- Linting/formatting: ESLint (typescript-eslint) + Prettier, enforced via Husky pre-commit hooks and lint-staged.
- Testing: Vitest for unit/integration tests; Playwright for dashboard E2E tests (Phase 13+).

Technical Specification — Root tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "declaration": true,
    "composite": true,
    "incremental": true
  }
}
```

0.3 Setup Ollama Locally

- Install Ollama (ollama.com) on the dev machine or a dedicated inference host.
- Pull initial models: `ollama pull qwen2.5-coder:7b` and `ollama pull deepseek-coder-v2:16b` (or smaller quantized variants for constrained hardware, e.g. `qwen2.5-coder:3b`).
- Confirm local REST endpoint is reachable at `http://localhost:11434`.

- Decide model-routing policy: small/fast model for classification tasks, larger model for README/summary generation (formalized in Phase 5).

Technical Specification — Ollama Health Check

```
GET http://localhost:11434/api/tags
→ 200 OK { "models": [ { "name": "qwen2.5-coder:7b", ... } ] }
```

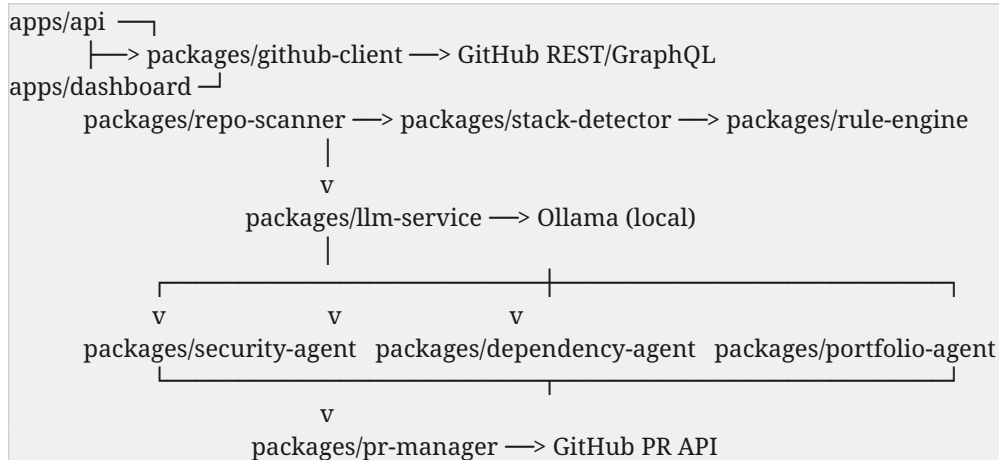
0.4 Design Database Schema for Repository Metadata

- Database choice: PostgreSQL (via Docker Compose for local dev) for relational integrity across repos, findings, and scores; SQLite acceptable for a single-user local-only mode.
- ORM: Prisma — chosen for type-safe schema-to-TypeScript generation, migrations, and good DX with monorepos.
- Design entities up front: Repository, Finding, StackProfile, HealthScore, Run, Agent, PullRequestRecord (core schema delivered in Phase 1 and Phase 2 specs below; remaining entities are expanded in later phases).

0.5 Design Modular Architecture

- Adopt a packages/apps monorepo layout (full tree in the Folder Structure appendix).
- Each capability (github-client, repo-scanner, stack-detector, rule-engine, llm-service, security-agent, dependency-agent, portfolio-agent, pr-manager) is an independent, independently-testable package with its own package.json and exported public API.
- Apps (`dashboard`, `api`) consume packages — never the reverse — to avoid circular dependencies.
- Define a shared `packages/core-types` package early so every other package shares one source of truth for types like `RepoMetadata`, `Finding`, `Severity`.

Architecture Diagram (textual)



Phase 0 — Acceptance Criteria

- ✓ `docs/scope.md` exists and is reviewed.
- ✓ Monorepo boots with `pnpm install && pnpm build` with zero errors across empty packages.
- ✓ Ollama responds to a local `/api/generate` test call with a real model.
- ✓ Prisma schema file compiles and produces a migration against a local Postgres instance.
- ✓ Architecture diagram and folder structure are committed to `docs/architecture.md`.

Phase 1 — Repository Discovery

Estimated Duration	1.5 weeks	Dependencies	Phase 0
--------------------	-----------	--------------	---------

Builds the GitHub-facing data layer: authenticated access, repository enumeration, and persistence of core metadata that every later phase reads from.

1.1 Connect to GitHub API

- Auth strategy: GitHub App (preferred for org-wide installs, higher rate limits) with fallback to Personal Access Token (classic, fine-grained) for individual/local use.
- Use Octokit (`@octokit/rest` + `@octokit/graphql`) as the typed client, wrapped inside `packages/github-client` so no other package talks to Octokit directly.
- Implement token storage via environment variables in local mode; via encrypted secrets table (using `@noble/ciphers` AES-GCM) in multi-user/dashboard mode.
- Implement rate-limit-aware request wrapper: read `x-ratelimit-remaining` / `x-ratelimit-reset` headers, queue and backoff automatically (prefer a small custom token-bucket limiter, or use `bottleneck` only if its maintenance and dependency risk are accepted).

Technical Specification — github-client Public API

```
interface GithubClient {
  listUserRepos(opts?: { affiliation?: string; perPage?: number }): Promise<RepoSummary[]>;
  listOrgRepos(org: string): Promise<RepoSummary[]>;
  getRepoTree(owner: string, repo: string, ref?: string): Promise<TreeEntry[]>;
  getFileContent(owner: string, repo: string, path: string): Promise<string | null>;
  getRateLimitStatus(): Promise<RateLimitStatus>;
}
```

1.2 Fetch Repositories

- Use GraphQL `viewer.repositories` query (paginated, 100/page) instead of REST for fewer round-trips when bulk-fetching star/fork/topic data simultaneously.
- Support filters: include/exclude forks, include/exclude archived repos, minimum-stars threshold, topic allow-list.
- Implement incremental sync: store `pushed\_at` per repo; on subsequent runs only re-fetch repos where `pushed\_at` changed.

Technical Specification — GraphQL Query

```
query ListRepos($cursor: String) {
  viewer {
    repositories(first: 100, after: $cursor, ownerAffiliations: [OWNER]) {
      pageInfo { hasNextPage endCursor }
      nodes {
        name nameWithOwner isFork isArchived
        stargazerCount forkCount pushedAt updatedAt
        primaryLanguage { name }
        repositoryTopics(first: 10) { nodes { topic { name } } }
      }
    }
  }
}
```

1.3 Store Metadata Locally

- Persist via Prisma into the `Repository` table (schema below) on every discovery run.
- Upsert strategy keyed on `nameWithOwner` to avoid duplicate rows across repeated scans.
- Store a `Run` record per discovery execution for audit/history (links forward to Phase 13 analytics).

Technical Specification — Prisma Schema (Repository + Run)

```
model Repository {
  id          String @id @default(uuid())
  nameWithOwner String @unique
  name        String
  owner       String
  isFork      Boolean @default(false)
  isArchived  Boolean @default(false)
  stars       Int    @default(0)
  forks       Int    @default(0)
  primaryLanguage String?
  topics      String[]
  pushedAt    DateTime
  lastScannedAt DateTime?
  findings    Finding[]
  stackProfile StackProfile?
  healthScores HealthScore[]
  createdAt   DateTime @default(now())
}

model Run {
  id          String @id @default(uuid())
  type        String // "discovery" | "audit" | "stack-detect" | "full-scan"
  startedAt   DateTime @default(now())
  finishedAt  DateTime?
  status      String // "running" | "success" | "failed"
  repoCount   Int     @default(0)
}
```

1.4 Track Stars, Forks, Topics, and Last Update Dates

- Expose a `repo-scanner` aggregation function that computes derived signals: days-since-last-push, star velocity (vs. previous snapshot), topic coverage (has topics vs. none).
- These derived signals feed directly into the Health Score formula (Phase 2) and the Portfolio Optimizer (Phase 10).

Technical Specification — Derived Metrics

Metric	Formula / Source	Used In
daysSinceLastPush	now - pushedAt (days)	Health Score (Phase 2)
starVelocity	(stars_now - stars_prev) / days_between_runs	Portfolio Optimizer
hasTopics	topics.length > 0	Portfolio Optimizer
isStale	daysSinceLastPush > 180	Health Score, Dashboard flags

Phase 1 — Acceptance Criteria

- ✓ Running the discovery CLI/command populates the `Repository` table for the authenticated user/org.
- ✓ Re-running discovery does not create duplicate rows (upsert verified).
- ✓ Rate-limit wrapper correctly pauses and resumes when GitHub's remaining quota nears zero (tested with a mock low-quota response).
- ✓ A `Run` row is created and marked `success` or `failed` for every discovery execution.

Phase 2 — Deterministic Repository Auditor

Estimated Duration	2 weeks	Dependencies	Phase 1
--------------------	---------	--------------	---------

Runs fast, rule-based (non-AI) checks against each repository's file tree to surface hygiene and exposure issues, and produces the first version of the repository Health Score.

2.1 Detect README, LICENSE, .gitignore

- Fetch root-level tree via ``getRepoTree`` (Phase 1 client) and check for case-insensitive matches: ``README*`, `LICENSE*`, `.gitignore``.
- For README, additionally check minimum content length (>200 chars) to flag "stub" READMEs, not just presence/absence.
- For LICENSE, attempt to classify license type via SPDX matching against known license texts (use the ``spdx-license-list`` npm package for matching, no AI needed).

2.2 Detect node_modules Uploads

- Walk the repo tree for any path segment literally named ``node_modules`, `vendor`, `.venv`, or `__pycache__`` committed to git (not just `.gitignored`).
- Cross-check against `.gitignore`` contents — flag as Critical if such a folder exists in the tree AND is absent from `.gitignore``.

2.3 Detect .env Exposure

- Search tree for filenames matching `.env`, `.env.*`` (excluding `.env.example`, `.env.sample`, `.env.template``).
- If found, fetch file content and run a lightweight entropy/regex secret scan (full implementation deferred to the Security Agent in Phase 8; here we only flag presence, not contents).
- Severity: Critical if `.env`` is tracked and not gitignored; Medium if tracked but added to `.gitignore`` in a more recent commit (meaning history still holds it).

2.4 Detect Large Files

- Use the Git Trees API with ``recursive=1`` to get blob sizes without cloning.
- Threshold: flag files >5MB as Medium, >25MB as High, >95MB (near GitHub's 100MB hard limit) as Critical.
- Recommend Git LFS migration in the finding's remediation text for binary asset types (images, video, model weights, datasets).

2.5 Generate Repository Health Scores

- Health Score is a weighted composite 0–100, computed deterministically from the findings above plus the staleness signal from Phase 1.
- Score is stored per run (not just latest) to support the historical trend analytics built in Phase 13.

Technical Specification — Finding Schema (Prisma)

```
model Finding {
  id          String @id @default(uuid())
  repositoryId String
  repository   Repository @relation(fields: [repositoryId], references: [id])
}
```

```

category String // "hygiene" | "security" | "dependency" | "stack" | "structure"
ruleId String // e.g. "missing-readme", "env-exposed"
severity String // "Critical" | "High" | "Medium" | "Low" | "Info"
confidence Float // 0.0 - 1.0, deterministic rules default to 1.0
message String
filePath String?
detectedAt DateTime @default(now())
resolvedAt DateTime?
}

model HealthScore {
  id String @id @default(uuid())
  repositoryId String
  repository Repository @relation(fields: [repositoryId], references: [id])
  score Int // 0-100
  breakdown Json // { hygiene: 25, security: 30, dependency: 20, structure: 15, freshness: 10 }
  computedAt DateTime @default(now())
}

```

Technical Specification — Health Score Weighting

Category	Weight	Deductions Example
Hygiene (README/LICENSE/.gitignore)	25 pts	-10 missing README, -5 stub README, -5 missing LICENSE, -5 missing .gitignore
Security exposure	30 pts	-30 tracked .env with no .gitignore entry, -15 if added but still in history
Dependency health	20 pts	Populated in Phase 9; placeholder 20 until then
Structure (large files / node_modules tracked)	15 pts	-15 node_modules tracked, -10 file >95MB, -5 file >25MB
Freshness	10 pts	-10 if isStale (>180 days), -5 if 90–180 days

Phase 2 — Acceptance Criteria

- ✓ Auditor runs against a test repo with deliberately seeded issues (missing README, tracked .env, tracked node_modules, an 80MB file) and produces all four corresponding findings.
- ✓ Health Score for a "clean" reference repo computes to ≥ 90 .
- ✓ Findings persist correctly with severity and category populated.
- ✓ Re-running the auditor on an unchanged repo does not duplicate open findings (idempotent upsert by `ruleId` + `filePath`).

Phase 3 — Stack Detection Engine

Estimated Duration	2 weeks	Dependencies	Phase 1, Phase 2
--------------------	---------	--------------	------------------

Determines what a repository is actually built with — framework, language, database, deployment target — so that later phases (rule engine, LLM prompts, dependency intelligence) can apply stack-specific logic instead of generic guesses.

3.1 Identify Framework

- Primary signal: presence and contents of manifest files — ``package.json`` (dependencies/devDependencies), ``requirements.txt`/`pyproject.toml``, ``composer.json``, ``Gemfile``, ``pom.xml`/`build.gradle``, ``Cargo.toml``.
- Maintain a ``detectors/`` rule table mapping known dependency names to frameworks, e.g. ``next`` → Next.js, ``@nestjs/core`` → NestJS, ``django`` → Django, ``laravel/framework`` → Laravel.
- Fallback: if no manifest match, inspect file extensions and directory conventions (e.g. ``app/``, ``pages/``, ``src/main/java``) for a best-guess heuristic with lower confidence.

3.2 Identify Language

- Use GitHub's own ``languages`` GraphQL field as the primary source (byte-weighted breakdown) — no need to re-implement linguist.
- Cross-validate against manifest file type when languages are ambiguous (e.g., a repo that's 60% HTML is likely a static site, not necessarily a "web app").

3.3 Identify Database

- Scan manifests for known DB driver/ORM packages: ``pg``, ``mysql2``, ``mongoose``, ``prisma``, ``sqlalchemy``, ``psycopg2``, ``redis``.
- Scan for ``docker-compose.yml`` service definitions (``image: postgres``, ``image: mongo``, etc.) as a secondary, often more reliable signal.
- Scan ``.env.example`` for connection-string variable name patterns (``DATABASE_URL``, ``MONGO_URI``) without reading actual secret values.

3.4 Identify Deployment Platform

- Detect platform-specific config files: ``vercel.json`` → Vercel, ``netlify.toml`` → Netlify, ``Procfile`` → Heroku, ``Dockerfile`` + ``fly.toml`` → Fly.io, ``.github/workflows/*.yml`` containing ``aws-actions`` → AWS.
- If multiple signals conflict, record all candidates with confidence scores rather than forcing a single answer.

3.5 Generate Stack Profile

- Aggregate the four signals above into one ``StackProfile`` record per repository, with a confidence score per field so downstream consumers can decide whether to trust or re-verify via the LLM (Phase 5).

Technical Specification — StackProfile Schema

```
model StackProfile {
  id          String @id @default(uuid())
  repositoryId String @unique
  repository  Repository @relation(fields: [repositoryId], references: [id])
}
```

```

framework String?
frameworkConfidence Float @default(0)
language String?
database String?
databaseConfidence Float @default(0)
deployTarget String?
deployConfidence Float @default(0)
rawSignals Json // full evidence trail for debugging / re-scoring
detectedAt DateTime @default(now())
}

```

Technical Specification — Detector Rule Table (excerpt)

Signal	Pattern	Maps To	Confidence
package.json dep	"next"	Next.js	0.95
package.json dep	"express"	Express.js	0.85
requirements.txt	django	Django	0.95
docker-compose service	image: postgres*	PostgreSQL	0.9
root file	vercel.json	Vercel	0.9
root file	Procfile	Heroku	0.85

Phase 3 — Acceptance Criteria

- ✓ Stack detector correctly identifies framework/language/database/deploy target for 5 reference repos spanning Next.js, Django, Laravel, a plain Node/Express API, and a static site.
- ✓ Confidence scores are populated for every field, never null when any signal exists.
- ✓ Conflicting signals are recorded in `rawSignals`, not silently dropped.

Phase 4 — Rule Engine

Estimated Duration	2.5 weeks	Dependencies	Phase 3
--------------------	-----------	--------------	---------

Layers stack-aware, framework-specific checks on top of the generic auditor from Phase 2, producing richer, more actionable findings with severity and confidence.

4.1 Load Stack-Specific Rules

- Rules are defined as plain TypeScript modules implementing a common `Rule` interface, grouped by stack key (e.g. `rules/nextjs/*.ts`, `rules/django/*.ts`).
- The engine loads only the rule sets matching the repo's detected `StackProfile.framework`, plus a universal rule set that always runs.

Technical Specification — Rule Interface

```
interface Rule {
  id: string;
  appliesTo: string[]; // e.g. ["nextjs"], or ["*"] for universal
  category: "hygiene" | "security" | "performance" | "structure" | "best-practice";
  severity: "Critical" | "High" | "Medium" | "Low" | "Info";
  check(ctx: RepoContext): Promise<RuleResult[]>;
}

interface RuleResult {
  passed: boolean;
  message: string;
  filePath?: string;
  confidence: number; // 0-1
}
```

4.2 Run Framework-Specific Checks

- Examples for Next.js: flag mixed `pages/` and `app/` routing as an architecture-consistency / migration-review signal, not as a deprecation error; flag missing image remote-pattern configuration when `next/image` uses external URLs.
- Examples for Express/Node APIs: flag missing `helmet` middleware, flag absence of input validation library (`zod`/`joi`) despite request-body usage.
- Examples for Django: flag `DEBUG = True` left in a non-`.example` settings file, flag missing `ALLOWED_HOSTS` restriction.
- Each check is unit-testable in isolation against fixture repos checked into `packages/rule-engine/___fixtures___/`.

4.3 Generate Structured Findings

- Rule engine findings use the same `Finding` schema from Phase 2 — `category`, `ruleId`, `severity`, `confidence`, `message`, `filePath` — so the dashboard and Modernization Planner treat deterministic and rule-engine findings uniformly.
- Each finding includes a `remediation` text template with placeholders the Cleanup Engine (Phase 7) can later fill in or act on directly.

4.4 Assign Severity and Confidence

- Severity is fixed per rule definition (rules don't change severity at runtime) to keep scoring deterministic and explainable.
- Confidence reflects pattern-matching certainty, not importance — e.g. a regex-based detection of ``DEBUG = True`` is high confidence (0.95) even though it's a security rule, while a heuristic "this might be dead code" check might be confidence 0.5.

Technical Specification — Severity → Score Impact Mapping

Severity	Points Deducted	SLA Recommendation
Critical	-20	Fix immediately; blocks portfolio showcase eligibility (Phase 10)
High	-10	Fix within current sprint
Medium	-5	Fix within current quarter
Low	-2	Opportunistic / nice-to-have
Info	0	Informational only, no score impact

Phase 4 — Acceptance Criteria

- ✓ At least 8 framework-specific rules exist across at least 3 stacks (e.g. Next.js, Express, Django) with passing unit tests against fixtures.
- ✓ Rule engine output merges cleanly with Phase 2 findings in the dashboard without duplicate ``ruleId`` collisions.
- ✓ Disabling a stack's rule set (e.g. via config) causes zero findings from that set, verified by an integration test.

Phase 5 — Local LLM Integration

Estimated Duration	2.5 weeks	Dependencies	Phase 0, Phase 3
--------------------	-----------	--------------	------------------

Wraps Ollama in a typed service so all AI-generated content (summaries, READMEs, classifications) flows through one consistent, swappable interface — with zero reliance on any paid hosted API.

5.1 Integrate Ollama

- Build ``packages/llm-service`` as a thin typed wrapper over Ollama's ``/api/generate`` and ``/api/chat`` REST endpoints — no official SDK dependency required, just ``fetch``.
- Support streaming responses for the dashboard's live generation view, and a blocking mode for batch/CLI jobs.
- Add a request queue with concurrency limit (default 1–2 concurrent generations) tuned to available local VRAM/CPU.

Technical Specification — llm-service Public API

```
interface LlmService {
  generate(opts: {
    task: "summary" | "readme" | "classification" | "pr-description";
    model?: string; // override default routing
    prompt: string;
    context?: Record<string, unknown>;
    stream?: boolean;
  }): Promise<LlmResult>;
}

interface LlmResult {
  text: string;
  model: string;
  durationMs: number;
  promptTokens?: number;
  responseTokens?: number;
}
```

5.2 Use Qwen or DeepSeek Models

- Default routing policy: ``qwen2.5-coder:3b`` for fast classification tasks (repo type, primary intent); ``qwen2.5-coder:7b`` or ``deepseek-coder-v2:16b`` for longer-form generation (README, summaries) where quality matters more than latency.
- Make model choice configurable per task via ``config/llm-routing.json`` so users on lower-spec hardware can downgrade everything to the smallest model.

Technical Specification — Model Routing Config

```
{
  "classification": "qwen2.5-coder:3b",
  "summary": "qwen2.5-coder:7b",
  "readme": "deepseek-coder-v2:16b",
  "prDescription": "qwen2.5-coder:7b",
  "fallback": "qwen2.5-coder:3b"
}
```


5.3 Generate Repository Summaries

- Prompt template pulls from the StackProfile (Phase 3) and top-level file tree to ground the summary in actual detected facts, reducing hallucination risk.
- Summaries are capped at ~150 words and stored alongside the repo for reuse in both the dashboard and the Portfolio Optimizer (Phase 10).

5.4 Generate README Files

- README generation uses a structured prompt requesting specific sections (Overview, Tech Stack, Getting Started, Scripts, License) rather than free-form text, then validates the output contains each expected heading before accepting it.
- If a README already exists, the LLM is given the existing content and asked to produce a diff-style improvement rather than a full rewrite, preserving any human-authored sections.
- Generated READMEs are never auto-committed directly — they flow into the Cleanup Engine (Phase 7) as a proposed change requiring human approval via PR (Phase 12).

5.5 Classify Repository Type

- Classification taxonomy: `web-app`, `api-service`, `cli-tool`, `library`, `data-science`, `learning-project`, `template`, `archived-experiment`.
- Classification combines deterministic signals (StackProfile, presence of `bin/` or `setup.py` entry points) with an LLM tie-breaker only when deterministic confidence is below 0.6.

Technical Specification — Prompt Template (Summary)

SYSTEM: You are a precise technical writer. Only state facts present in the provided context. Do not invent features, libraries, or claims.

CONTEXT:

Framework: {{framework}}
 Language: {{language}}
 Database: {{database}}
 File tree (top 30 paths): {{tree}}

TASK: Write a 2-3 sentence summary of what this repository does and the technology it uses. Be factual and concise.

Phase 5 — Acceptance Criteria

- ✓ `llm-service` successfully calls a local Ollama model and returns text for all four task types.
- ✓ Generated READMEs contain all required section headings on at least 90% of test runs across 10 sample repos.
- ✓ No network calls are made to any non-localhost LLM endpoint — verified via a network-egress test/log audit.
- ✓ Classification accuracy ≥80% against a manually labeled set of 25 reference repos.

Phase 6 — Modernization Planner

Estimated Duration	2 weeks	Dependencies	Phase 4, Phase 5
--------------------	---------	--------------	------------------

Converts the raw findings pile (Phases 2 and 4) into a prioritized, effort-aware action plan per repository — the bridge between "what's wrong" and "what to do about it."

6.1 Convert Findings into Action Plans

- Group related findings into a single `ActionItem` (e.g. three separate "missing X" hygiene findings become one "Repository Hygiene Cleanup" action item) to avoid overwhelming the user with noise.
- Each ActionItem carries the list of source `Finding` IDs it resolves, so completion can be tracked precisely.

Technical Specification — ActionItem Schema

```
model ActionItem {
  id      String @id @default(uuid())
  repositoryId String
  title    String
  description String
  category String
  priority Int // 1 (highest) - 5 (lowest), computed in 6.2
  effortEstimate String // "XS" | "S" | "M" | "L" | "XL"
  findingIds String[]
  status    String @default("open") // open | in-progress | done | dismissed
  createdAt DateTime @default(now())
}
```

6.2 Prioritize Fixes

- Priority score = severity weight (from Phase 4 table) × confidence, with a bonus multiplier for Critical security items so they always surface first regardless of effort.
- Ties are broken by effort (smaller effort wins) to surface quick wins ahead of large refactors of similar severity.

Technical Specification — Priority Formula

```
priorityScore = (severityWeight * confidence) + securityBonus - (effortPenalty)
```

```
severityWeight: Critical=100, High=60, Medium=30, Low=10, Info=0
```

```
securityBonus: +50 if category === "security" and severity in [Critical, High]
```

```
effortPenalty: XS=0, S=2, M=5, L=10, XL=20
```

6.3 Estimate Effort

- Effort is estimated heuristically per rule type (not via LLM, to keep it deterministic and auditable) — e.g. "add LICENSE" is always XS, "migrate pages/ to app/ router" is always L/XL.
- Each rule definition (Phase 4) is extended with a static `defaultEffort` field; the planner reads this directly rather than re-deriving it.

6.4 Generate Modernization Roadmap per Repository

- Roadmap groups ActionItems into themed milestones: "Week 1 — Security & Hygiene", "Week 2 — Dependency Updates", "Week 3+ — Structural Modernization".

- Roadmap is rendered both as a dashboard Kanban-style view and as a Markdown export (`ROADMAP.md`) that can be committed to the repo itself or attached to the generated PR.

Technical Specification — Roadmap Output Shape

```
interface Roadmap {  
  repositoryId: string;  
  generatedAt: string;  
  milestones: {  
    name: string;  
    actionItemIds: string[];  
    estimatedEffort: string; // sum, e.g. "3 days"  
  }[];  
}
```

Phase 6 — Acceptance Criteria

- ✓ Given a repo with 12 mixed-severity findings, the planner produces a roadmap with no more than 6 grouped ActionItems.
- ✓ Critical security ActionItems always appear in the first milestone regardless of effort size.
- ✓ `ROADMAP.md` export renders correctly as valid Markdown and matches the dashboard view 1:1.

Phase 7 — Cleanup Engine

Estimated Duration	2 weeks	Dependencies	Phase 6
--------------------	---------	--------------	---------

Turns approved ActionItems into actual file changes — generated content and shell-level cleanup commands — staged and ready for the PR Automation system in Phase 12.

7.1 Generate Cleanup Commands

- For structural findings (tracked `node_modules``, large binaries), generate the exact `git rm --cached` / `git filter-repo`` command sequence needed, but never execute history-rewriting commands automatically — always require explicit human confirmation given the destructive/irreversible nature of history rewrites.
- Commands are rendered into a `cleanup-plan.sh`` artifact per repo, reviewed by the user before any execution.

7.2 Create .gitignore

- Use the `github/gitignore`` template repository as the base, selected automatically by detected language/framework (Phase 3 StackProfile).
- Merge with any existing `.gitignore`` content rather than overwriting, de-duplicating lines.

7.3 Generate LICENSE

- Default to MIT unless the user's GitHub profile or org has a stated preference; expose a config option for Apache-2.0, GPL-3.0, or "All Rights Reserved" (private/proprietary template).
- Auto-fill copyright year (current year) and holder name (from GitHub profile `name`` field, falling back to username).

7.4 Generate README

- Pulls directly from the LLM service (Phase 5) output, post-processed to insert standard badges (build status, license, last commit) using shields.io URL templates — generated locally, no external AI needed for badge URLs.

7.5 Prepare Pull Requests

- Cleanup Engine's output is a `ChangeSet``: a set of file additions/modifications plus optional shell commands, scoped to one repository, ready to hand to `pr-manager`` (Phase 12).
- Each ChangeSet is content-hashed so re-running cleanup on an unchanged repo is a no-op (idempotency).

Technical Specification — ChangeSet Schema

```
interface ChangeSet {
  repositoryId: string;
  actionItemIds: string[];
  fileChanges: {
    path: string;
    action: "create" | "modify" | "delete";
    content?: string; // for create/modify
  }[];
  shellCommands?: string[]; // require manual confirmation, never auto-run
```

```
contentHash: string;  
}
```

Phase 7 — Acceptance Criteria

- ✓ Running cleanup on a repo missing README/LICENSE/.gitignore produces a ChangeSet with all three files correctly generated and stack-appropriate.
- ✓ No destructive git history command is ever executed without an explicit separate confirmation step.
- ✓ Re-running cleanup on an already-clean repo produces an empty ChangeSet (verified idempotency test).

Phase 8 — Security Agent

Estimated Duration	2 weeks	Dependencies	Phase 2, Phase 3
--------------------	---------	--------------	------------------

Deepens the lightweight .env-presence check from Phase 2 into a full secret-scanning and exposure-reporting subsystem. Can be developed in parallel with Phases 4–6 since it only depends on the auditor and stack detector.

8.1 Scan for Secrets

- Integrate a regex + entropy-based scanner (build on patterns from `gitleaks`/`trufflehog` rule sets, reimplemented locally rather than shelling out, to keep the dependency surface auditable) covering AWS keys, generic API keys, JWTs, private key blocks (`-----BEGIN PRIVATE KEY-----`), and database connection strings with embedded credentials.
- Scan both current file content and, optionally, full commit history (opt-in due to cost) using the Git Trees API across historical blobs.
- Never log or persist the actual secret value — only its location (file path, line number) and a redacted preview (first/last 2 characters).

Technical Specification — SecretFinding Schema

```
model SecretFinding {
  id      String @id @default(uuid())
  repositoryId String
  filePath String
  lineNumber Int
  ruleId   String // e.g. "aws-access-key", "private-key-block"
  redactedPreview String // e.g. "AKIA*****XYZ"
  severity String @default("Critical")
  inHistoryOnly Boolean @default(false) // true if not in current HEAD, only past commits
  detectedAt DateTime @default(now())
}
```

8.2 Scan for Exposed API Keys

- Maintain a pattern library keyed by provider (Stripe `sk\_live\_`, SendGrid `SG.`, Slack `xox[baprs]-`, Google API `Alza`, etc.) for higher-confidence, named findings rather than generic "looks like a secret" noise.
- Each match includes a confidence score; generic high-entropy strings get lower confidence than provider-pattern matches to reduce false positives.

8.3 Scan Environment Files

- Specifically diff `.env` against `.env.example` to flag variables present in the real file but absent from the example template (signals real secrets that should be documented, not committed).
- Flag `.env` files committed in any historical commit even if removed from HEAD — git history retains the blob until a full rewrite.

8.4 Generate Security Reports

- Produce a per-repository `SecurityReport` summarizing counts by severity, affected files, and a recommended remediation order (rotate the credential first, then scrub history, then add prevention via `.gitignore`/pre-commit hook).

- Reports are exportable as PDF (using the same LibreOffice conversion path used for docx) for sharing with a team or auditor.

Technical Specification — Remediation Priority

Step	Action	Why First
1	Rotate/revoke the exposed credential at the provider	The repo's git history is permanent; rotation neutralizes risk immediately
2	Remove from current HEAD via `git rm` + commit	Stops further exposure on default branch
3	Scrub history with `git filter-repo` (manual confirm required)	Removes from clone-able history; destructive, needs explicit approval
4	Add `.gitignore` entry + pre-commit secret-scan hook	Prevents recurrence

Phase 8 — Acceptance Criteria

- ✓ Scanner correctly identifies seeded test secrets (AWS key pattern, Stripe key pattern, generic high-entropy string) in a fixture repo with zero false negatives on the named patterns.
- ✓ No raw secret value is ever written to logs, database, or reports — verified via a grep audit of generated artifacts.
- ✓ Security report PDF generates successfully and lists findings in the documented remediation order.

Phase 9 — Dependency Intelligence

Estimated Duration	2 weeks	Dependencies	Phase 3
--------------------	---------	--------------	---------

Fills in the placeholder dependency-health score from Phase 2 with real outdated/unused-package analysis. Can also run in parallel with Phases 4–8.

9.1 Detect Outdated Packages

- For Node projects: run ``npm outdated --json`` (or parse ``package.json`` against the npm registry API directly to avoid requiring a full ``npm install``) to get current vs. latest vs. wanted versions.
- For Python: compare ``requirements.txt`/`pyproject.toml`` pins against PyPI's JSON API (``https://pypi.org/pypi/{package}/json``).
- Classify each outdated dependency by semver distance: patch-behind, minor-behind, major-behind — major-behind carries higher priority in the planner (Phase 6) due to likely breaking changes.

9.2 Find Unused Dependencies

- Use static analysis (``depcheck`` for Node, or a custom AST-based import scanner) to detect packages declared in the manifest but never imported anywhere in source.
- Exclude known false-positive categories: type-only packages (``@types/*``), build tool plugins referenced only in config files, peer dependencies.

9.3 Suggest Upgrades

- For major-version-behind packages, check whether the package publishes a migration guide (common changelog/readme heading patterns) and surface a link in the finding rather than attempting automated codemods in v1.
- Where official codemods exist and are well-established (e.g. React's ``react-codemod``, Next.js's ``@next/codemod``), offer them as an optional one-click action wired through the Cleanup Engine's shell-command mechanism (Phase 7), still requiring confirmation.

9.4 Generate Dependency Health Score

- Formalizes the 20-point placeholder category from Phase 2's Health Score: deduct proportionally to count and severity of outdated/unused packages, with major-behind weighted heavier than minor/patch.

Technical Specification — DependencyFinding Schema

```
model DependencyFinding {
  id      String @id @default(uuid())
  repositoryId String
  packageName String
  currentVersion String
  latestVersion String
  semverDistance String // "patch" | "minor" | "major"
  status    String // "outdated" | "unused" | "deprecated"
  registryUrl String?
  detectedAt DateTime @default(now())
}
```


Technical Specification — Dependency Score Deduction

deduction = sum(packageDeductions), capped at 20
major-behind: -4 per package (max 3 counted)
minor-behind: -1.5 per package (max 4 counted)
unused: -1 per package (max 4 counted)
deprecated (npm "deprecated" flag): -5 flat, regardless of version gap

Phase 9 — Acceptance Criteria

- ✓ Outdated-package scan correctly classifies semver distance for a fixture `package.json` with known patch/minor/major-behind packages.
- ✓ Unused-dependency detection produces zero false positives against a fixture repo containing common edge cases (`@types/\*`, a Tailwind config-only plugin).
- ✓ Dependency Health Score sub-component replaces the Phase 2 placeholder cleanly with no regression to the rest of the composite score.

Phase 10 — Portfolio Optimizer

Estimated Duration	1.5 weeks	Dependencies	Phase 5, Phase 6
--------------------	-----------	--------------	------------------

Shifts focus from "fix what's broken" to "present what's good" — generating GitHub-facing metadata and surfacing which repositories are strong enough to showcase to recruiters, clients, or collaborators.

10.1 Generate GitHub Descriptions

- Use the LLM service (Phase 5) with a tighter prompt constraint: GitHub repo descriptions are capped at 350 characters, so the prompt explicitly enforces a target of under 120 characters for readability in repo list views.
- Description generation pulls from the Stack Profile and the repo summary, but explicitly excludes any marketing-style superlatives ("amazing", "revolutionary") via a post-generation filter — factual tone only.

10.2 Generate Repository Topics

- Suggest 5–8 GitHub topics combining: detected language/framework (deterministic, high confidence), detected repo type/classification from Phase 5, and up to 2 LLM-suggested domain topics (e.g. "e-commerce", "data-visualization").
- Cross-check suggested topics against GitHub's most popular topic list (via the Topics search API) to prefer canonical spellings (`nextjs` not `next-js`).

10.3 Generate Portfolio Summaries

- Produces a longer-form (3–5 sentence) summary suited for a personal portfolio site or resume context, distinct from the short repo-card summary in Phase 5 — same underlying facts, different register and length.

10.4 Recommend Showcase Projects

- Showcase eligibility formula combines: Health Score (Phase 2, must be ≥ 80), recency (pushed within 12 months OR explicitly marked "maintained"), and presence of a complete README with a live-demo or screenshot link.
- Output a ranked top-N list (configurable, default 6) intended to populate a pinned-repos selection or personal site project grid.

Technical Specification — Showcase Score

```
showcaseEligible = healthScore >= 80
                  && (daysSinceLastPush <= 365 || manuallyMarkedMaintained)
                  && hasCompleteReadme

showcaseRank = (healthScore * 0.5) + (starVelocityNormalized * 0.3) + (recencyScore * 0.2)
```

Phase 10 — Acceptance Criteria

- ✓ Generated descriptions stay under the configured character target for 100% of test runs (hard validation, not just prompt instruction).
- ✓ Suggested topics include at least the deterministic language/framework topics every time, with zero placeholder/empty topic slots.

- ✓ Showcase ranking produces a stable, reproducible ordering given identical input scores (no non-determinism from LLM calls in the ranking step itself, only in descriptive text).

Phase 11 — Multi-Agent Architecture

Estimated Duration	3 weeks	Dependencies	Phase 5 – Phase 10
--------------------	---------	--------------	--------------------

Refactors the previously independent capabilities (security, dependency, portfolio, etc.) into a coordinated agent framework with a shared orchestration layer, message contracts, and a documentation agent that didn't exist as a standalone unit before.

11.1 Security Agent (Formalized)

- Wraps Phase 8's secret-scanning and reporting logic behind the standard `Agent` interface (below), exposing `analyze(repo)` and `getFindings(repo)`.

Technical Specification — Agent Interface

```
interface Agent {
  name: string;
  version: string;
  analyze(ctx: RepoContext): Promise<AgentResult>;
  dependsOn?: string[]; // other agent names whose output this agent reads
}

interface AgentResult {
  agentName: string;
  findings: Finding[];
  artifacts?: Record<string, unknown>; // e.g. generated README text, scores
  durationMs: number;
  status: "success" | "partial" | "failed";
  error?: string;
}
```

11.2 Documentation Agent (New)

- Consolidates README generation (Phase 5), ROADMAP.md export (Phase 6), and the security report (Phase 8) into one agent responsible for all generated-document output, ensuring consistent tone, formatting, and a single place to manage document templates.
- Owns the `docs-template` package containing shared Markdown partials (badges, footer, table-of-contents generation) reused across every generated document type.

11.3 Architecture Agent (New)

- New capability that synthesizes the Stack Profile (Phase 3) and Rule Engine findings (Phase 4) into a higher-level architectural assessment: monolith vs. modular, presence/absence of a service layer, test coverage posture (via presence and count of test files relative to source files).
- Surfaces "structural debt" findings distinct from simple hygiene findings — e.g. "controller files directly query the database with no repository/service abstraction" — detected via directory-convention heuristics rather than full static analysis in v1.

11.4 Portfolio Agent (Formalized)

- Wraps Phase 10 logic behind the `Agent` interface; declares `dependsOn: ["documentation-agent"]` since it needs the generated summary before producing showcase copy.

11.5 Dependency Agent (Formalized)

- Wraps Phase 9 logic behind the `Agent` interface; runs independently with no inter-agent dependency, allowing it to execute in parallel with the Security Agent.

11.6 Orchestration Layer

- Build an `AgentRunner` that resolves the dependency graph (via `dependsOn`) and executes agents in topological order, running independent agents concurrently via `Promise.all` batches.
- Each agent run is wrapped in a timeout and try/catch so one failing agent (e.g. Ollama briefly unavailable) doesn't abort the entire run — failed agents report `status: "failed"` and the run continues, surfaced clearly in the dashboard.

Technical Specification — Orchestration Flow

```
Batch 1 (parallel): SecurityAgent, DependencyAgent, ArchitectureAgent
Batch 2 (parallel): DocumentationAgent // depends on Architecture + Security findings
Batch 3:           PortfolioAgent    // depends on DocumentationAgent
```

Technical Specification — AgentRun Schema

```
model AgentRun {
  id      String @id @default(uuid())
  runId    String // links to the parent Run (Phase 1)
  agentName String
  status   String
  durationMs Int
  errorMessage String?
  startedAt DateTime @default(now())
}
```

Phase 11 — Acceptance Criteria

- ✓ All five agents conform to the shared `Agent` interface and pass a contract test verifying they return a valid `AgentResult` shape.
- ✓ Orchestrator correctly parallelizes independent agents (verified via timing logs showing overlapping `startedAt` windows for Security and Dependency agents).
- ✓ A simulated Ollama outage causes only the Documentation/Portfolio agents to report `failed`/`partial`, while Security and Dependency agents complete normally.

Phase 12 — Pull Request Automation

Estimated Duration	2.5 weeks	Dependencies	Phase 7, Phase 11
--------------------	-----------	--------------	-------------------

Closes the loop: takes the ChangeSets produced by the Cleanup Engine and the documents produced by the Documentation Agent, and turns them into real, reviewable GitHub pull requests — always gated by human approval.

12.1 Create Fix Branches

- Branch naming convention: `repo-modernizer/{actionItemCategory}-{shortHash}` (e.g. `repo-modernizer/hygiene-a1b2c3`) to keep automated branches clearly distinguishable and groupable.
- Branches are created from the latest default-branch SHA at execution time via the Git Data API (`createRef`), avoiding local clone/checkout entirely for speed.

12.2 Generate Commits

- Use the Git Data API tree/blob/commit endpoints to construct commits directly server-side (no local git binary dependency), supporting multi-file commits in a single API round-trip.
- Commit messages follow Conventional Commits format (`fix:`, `docs:`, `chore:`, `security:`) derived from the ActionItem category, improving changelog generation for any repo that already uses semantic-release.

Technical Specification — Commit Construction Flow

```
1. getRef(baseBranch) → baseSha
2. for each fileChange: createBlob(content) → blobSha
3. createTree(baseTreeSha, [{ path, sha: blobSha, mode: "100644" }])
4. createCommit(treeSha, [baseSha], message)
5. updateRef(branchRef, newCommitSha)
```

12.3 Open PRs Automatically

- PR body is generated by the Documentation Agent (Phase 11) and includes: a summary of what changed, the list of resolved Finding IDs, the before/after Health Score delta, and a checklist for the human reviewer.
- PRs are labeled automatically (`automated`, plus a category label like `security` or `dependencies`) for easy filtering in GitHub's UI.

Technical Specification — PR Body Template

```
## Automated Modernization: {{actionItemTitle}}

**Health Score Impact:** {{scoreBefore}} → {{scoreAfter}} (+{{delta}})

### Changes
{{fileChangeSummary}}

### Resolves
{{#each findingIds}}- Finding #{{this}}{/each}

### Reviewer Checklist
- [ ] Verify generated content accuracy
- [ ] Confirm no unintended file changes
```

- [] Run CI before merging

12.4 Allow Human Approval

- The system never merges its own PRs. Merge remains a manual GitHub action by the repository owner/maintainer, full stop — this is a hard product boundary, not a configurable toggle, to keep trust high.
- Track PR outcome (merged / closed-without-merge / still-open) back into the `PullRequestRecord` table to feed the Historical Analytics in Phase 13 (e.g. "% of automated PRs accepted").

Technical Specification — PullRequestRecord Schema

```
model PullRequestRecord {
  id      String @id @default(uuid())
  repositoryId String
  actionItemId String
  prNumber Int
  prUrl    String
  status   String // "open" | "merged" | "closed"
  scoreBefore Int
  scoreAfter Int?
  openedAt DateTime @default(now())
  resolvedAt DateTime?
}
```

Phase 12 — Acceptance Criteria

- ✓ End-to-end test: a fixture repo with seeded issues produces a branch, commit, and open PR with correctly populated body and labels, with zero local git CLI usage.
- ✓ No code path exists that calls GitHub's merge endpoint — verified by an explicit static check / lint rule banning the merge API call in the codebase.
- ✓ `PullRequestRecord.status` correctly transitions to `merged` or `closed` when a webhook or polling check detects the PR's resolution.

Phase 13 — Historical Analytics

Estimated Duration	2 weeks	Dependencies	Phase 2, Phase 12
--------------------	---------	--------------	-------------------

Turns the accumulating Run, HealthScore, and PullRequestRecord history into trend visibility — proving the system's value over time and identifying repos that are improving versus stagnating.

13.1 Track Repository Scores

- Every audit run already writes a new `HealthScore` row (Phase 2) rather than overwriting — this phase is primarily about querying and presenting that existing time series, not new collection logic.
- Add a materialized/cached `latestScore` denormalized field on `Repository` for fast dashboard list rendering without a join+sort on every page load.

13.2 Measure Improvements

- Compute score delta over configurable windows (7-day, 30-day, all-time) per repository and in aggregate across the whole portfolio.
- Compute "automation impact": average score delta attributable specifically to merged automated PRs (joining `PullRequestRecord.scoreBefore/scoreAfter`) versus delta from manual commits, to demonstrate the tool's direct contribution.

Technical Specification — Analytics Query Shapes

```
-- 30-day score trend per repo
SELECT repositoryId, computedAt, score
FROM HealthScore
WHERE computedAt >= now() - interval '30 days'
ORDER BY repositoryId, computedAt;

-- Automation impact aggregate
SELECT AVG(scoreAfter - scoreBefore) AS avgDelta, COUNT(*) AS mergedPrCount
FROM PullRequestRecord
WHERE status = 'merged' AND scoreAfter IS NOT NULL;
```

13.3 Visualize Progress Over Time

- Dashboard charts: per-repo score line chart, portfolio-wide average score trend, findings-resolved-over-time bar chart, and a leaderboard of most-improved repos this month.
- Charting library: Recharts (already standard in the dashboard's React stack) fed by a dedicated `apps/api` analytics endpoint rather than raw client-side DB queries.

Technical Specification — Analytics API Contract

```
GET /api/analytics/repo/:id/trend?window=30d
→ { repositoryId, points: [{ date, score }], delta, deltaPercent }

GET /api/analytics/portfolio/summary
→ { avgScore, totalRepos, repoCountAbove80, mostImproved: [{ repoId, delta }] }
```

Phase 13 — Acceptance Criteria

- ✓ Trend endpoint returns correct, chronologically ordered score points for a repo with at least 5 historical audit runs.

- ✓ Automation-impact aggregate correctly excludes non-merged PRs and PRs lacking a `scoreAfter` value.
- ✓ Dashboard renders the portfolio summary and per-repo trend chart without N+1 query patterns (verified via query-count assertion in integration tests).

Phase 14 — Continuous Monitoring

Estimated Duration	2 weeks	Dependencies	Phase 12, Phase 13
--------------------	---------	--------------	--------------------

Moves the platform from an on-demand tool to an always-watching system: scheduled scans, automated issue creation, and recurring reports delivered without manual triggering.

14.1 GitHub Actions Integration

- Publish a reusable composite GitHub Action (`repo-modernizer/scan-action`) that org members can drop into any repo's `.github/workflows/` to trigger a scan on push/PR or on a cron schedule, calling out to the hosted `apps/api` (or a fully local instance if self-hosted).
- Action outputs a job-summary markdown block showing the Health Score delta directly in the Actions tab, so results are visible without leaving GitHub.

Technical Specification — action.yml (excerpt)

```
name: "Repo Modernizer Scan"
inputs:
  api-url: { required: true }
  repo-id: { required: true }
runs:
  using: "node20"
  main: "dist/index.js"
```

14.2 Scheduled Scans

- Server-side scheduler (e.g. `node-cron` or a dedicated lightweight job queue like `BullMQ` backed by Redis) runs full audits on a per-repo cadence: daily for actively-developed repos (pushed within 30 days), weekly otherwise — avoiding wasted compute on dormant repos.
- Scheduling cadence is configurable per repository via a `scanFrequency` field, with sensible defaults applied automatically based on the staleness signal from Phase 1.

14.3 Weekly Reports

- Aggregate digest email/notification summarizing: portfolio-wide score trend (from Phase 13), new Critical/High findings this week, and any automated PRs opened or merged.
- Delivery channel is pluggable: email (via a local SMTP relay or a transactional provider), or a webhook payload formatted for Slack/Discord — kept provider-agnostic so no specific paid notification service is required.

14.4 Automated Issue Creation

- For Critical findings that aren't auto-fixable by the Cleanup Engine (e.g. "rotate this exposed credential" — a human-only action), automatically open a GitHub Issue rather than a PR, tagged `security` and assigned to the repo owner by default.
- De-duplicate issue creation against existing open issues referencing the same `ruleId` + `filePath` to avoid spamming the issue tracker on every scan.

Technical Specification — ScheduledScan Schema

```
model ScheduledScan {
  id      String  @id @default(uuid())
  repositoryId String @unique
```

```
frequency String // "daily" | "weekly" | "monthly"  
lastRunAt DateTime?  
nextRunAt DateTime  
enabled Boolean @default(true)  
}
```

Phase 14 — Acceptance Criteria

- ✓ The composite GitHub Action runs successfully in a sample workflow and produces a visible job summary.
- ✓ Scheduler correctly computes `nextRunAt` per repo according to its frequency tier, verified across daily/weekly/monthly fixtures.
- ✓ Re-scanning a repo with an already-open issue for the same finding does not create a duplicate issue.

Phase 15 — Dissertation / Research Extension

Estimated Duration	4–6 weeks	Dependencies	All prior phases
--------------------	-----------	--------------	------------------

Reframes the completed engineering platform as a research artifact — appropriate for a dissertation, thesis, or conference submission on automated software maintenance and AI-assisted code quality tooling.

15.1 Repository Quality Scoring Model

- Formalize the Health Score (Phases 2, 9) as a research contribution: document its weighting rationale, validate it against an external ground truth (e.g. correlate scores with repos later archived/abandoned versus actively maintained, or against existing tools like OpenSSF Scorecard).
- Run an inter-rater reliability style study: have human reviewers independently rate a sample of repos' "health" and compute correlation (Spearman's rho) against the automated score to validate construct validity.

15.2 Stack-Aware Prompt Generation

- Study how grounding LLM prompts in deterministic StackProfile facts (Phase 3/5) reduces hallucination versus a naive "summarize this repo" prompt with no structured context — ablation study comparing grounded vs. ungrounded prompts on factual-accuracy scoring.
- Document the model-routing policy (Phase 5) as a cost/quality/latency tradeoff study across model sizes (3B vs 7B vs 16B) on local hardware, useful as a practical reference for similar local-LLM tooling projects.

15.3 AI Modernization Planning

- Evaluate the Modernization Planner's (Phase 6) prioritization formula against expert-ranked priority orderings from experienced engineers reviewing the same finding sets, measuring rank correlation.
- Explore whether effort estimates (currently static per-rule heuristics) could be refined using historical PR cycle-time data collected in Phase 13 — a natural extension for a "future work" section.

15.4 Automated Software Maintenance Research

- Position the overall system within the existing literature on automated program repair, technical debt quantification, and bot-driven pull requests (e.g. Dependabot/Renovate comparisons), explicitly identifying what this system adds: stack-aware deterministic rules plus local-LLM-grounded documentation generation without external API dependency.
- Use the accumulated `PullRequestRecord` and `HealthScore` history (Phases 12–13) as the empirical dataset for the dissertation's results chapter — real before/after metrics from real usage, not synthetic benchmarks.

Suggested Research Deliverables

Deliverable	Source Data	Maps To
Health Score validation study	HealthScore history + manual ratings	15.1
Grounded vs. ungrounded prompt ablation	llm-service logs, Phase 5	15.2
Priority-ranking correlation study	ActionItem priority vs. expert	15.3

Deliverable	Source Data	Maps To
	review	
Literature review & positioning	External literature + system design docs	15.4
Empirical results chapter	PullRequestRecord + HealthScore tables	15.4

Phase 15 — Acceptance Criteria

- ✓ At least one quantitative validation study (e.g. Health Score correlation) is completed with a documented methodology and result.
- ✓ A written positioning section situates the system relative to at least 3 comparable existing tools or papers.
- ✓ Empirical results draw from real accumulated platform data (not solely simulated/synthetic data), per the project's own historical analytics.

Appendix A — Full Monorepo Folder Structure

Expanded from the original suggested structure with the concrete packages and files introduced across all 16 phases.

```

repo-modernizer/
├── apps/
│   ├── dashboard/           # Next.js dashboard (React, Recharts, Tailwind)
│   │   ├── app/
│   │   ├── components/
│   │   └── package.json
│   ├── api/                 # Express/Fastify API + scheduler + webhooks
│   │   ├── src/
│   │   │   ├── routes/
│   │   │   ├── analytics/    # Phase 13 endpoints
│   │   │   └── scheduler/    # Phase 14 cron jobs
│   │   └── package.json
│   └── packages/
│       ├── core-types/      # Shared TS types: RepoMetadata, Finding, Severity
│       ├── github-client/   # Phase 1 - Octokit wrapper, rate limiting
│       ├── repo-scanner/    # Phase 1/2 - tree walking, derived metrics
│       ├── stack-detector/  # Phase 3 - framework/lang/db/deploy detection
│       ├── rule-engine/     # Phase 4 - deterministic + framework rules
│       │   └── rules/
│       │       ├── nextjs/
│       │       ├── express/
│       │       └── django/
│       ├── llm-service/     # Phase 5 - Ollama wrapper, prompt templates
│       │   └── prompts/
│       ├── modernization-planner/ # Phase 6 - ActionItem + roadmap generation
│       ├── cleanup-engine/   # Phase 7 - ChangeSet generation
│       ├── security-agent/   # Phase 8/11 - secret scanning
│       ├── dependency-agent/ # Phase 9/11 - outdated/unused deps
│       ├── architecture-agent/ # Phase 11 - structural debt detection
│       ├── documentation-agent/ # Phase 11 - README/ROADMAP/report generation
│       ├── portfolio-agent/  # Phase 10/11 - descriptions, topics, showcase
│       ├── agent-runner/     # Phase 11 - orchestration layer
│       └── pr-manager/       # Phase 12 - branches, commits, PRs
├── database/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── migrations/
│   └── seed/
├── reports/                 # Generated PDF/Markdown security & health reports
├── docs/
│   ├── scope.md             # Phase 0
│   ├── architecture.md     # Phase 0
│   └── research/            # Phase 15 dissertation materials
├── .github/
│   └── workflows/
│       └── scan-action/     # Phase 14 composite action source

```

```
|  
├─ turbo.json  
├─ pnpm-workspace.yaml  
└─ package.json
```

Appendix B — Risk Register

Risk	Phase(s)	Likelihood	Mitigation
Local LLM hallucinates inaccurate README/summary content	5, 7, 11	Medium	Ground prompts in deterministic StackProfile facts; require human PR approval before merge (never auto-merge)
GitHub API rate limits throttle large-org scans	1, 12, 14	Medium	Token-bucket backoff, incremental sync by pushedAt, GitHub App install for higher limits
Secret scanner false negatives miss real exposure	8	High impact / Low likelihood	Combine provider-pattern + entropy detection; treat as a safety net, not sole control — still recommend credential rotation as default-first remediation
Destructive git history rewrite executed accidentally	7, 8	Low likelihood / High impact	Hard rule: history-rewriting commands are generated but never auto-executed; explicit separate human confirmation gate
Local hardware insufficient for 16B-class models	5	Medium	Config-driven model routing allows full downgrade to 3B models; document minimum/recommended hardware tiers
Agent orchestration cascading failure	11	Low	Per-agent timeout + isolated try/catch; partial-failure status surfaced rather than aborting the run
Scope creep from 16 phases delays MVP value	All	Medium	Phases 8 and 9 are parallelizable with 4–6; consider shipping Phases 0–7 as a usable MVP before continuing

Appendix C — Technology Stack Summary

Layer	Choice	Rationale
Language	TypeScript 6.0.x (strict)	Type safety across a multi-package monorepo with many shared contracts
Runtime	Node.js 24.x LTS recommended; Node.js 22.x LTS minimum	Long-term support, native fetch, broad ecosystem compatibility
Monorepo tooling	pnpm 11.x + Turborepo 2.9.x	Efficient workspace installs, build caching across 15+ packages
Database	PostgreSQL 18.x + Prisma 7.x	Relational integrity for findings/scores/runs; type-safe queries
LLM Runtime	Ollama (local)	Zero external API key dependency, full data locality
LLM Models	Qwen2.5-Coder, DeepSeek-Coder-V2	Strong code/doc generation quality at multiple size tiers for local hardware
GitHub Access	@octokit/rest 22.x + @octokit/graphql 9.x	Official, typed, well-maintained client
Dashboard	Next.js 16.x + React 19.x + Tailwind CSS + Recharts	Fast iteration, good charting ergonomics for Phase 13 analytics
Scheduling	node-cron / BullMQ + Redis	Reliable recurring jobs for Phase 14 continuous monitoring
Testing	Vitest 4.x + Playwright 1.61.x	Fast unit/integration tests; E2E coverage for dashboard flows

Appendix D - Required Technology Stack Versions

Version baseline checked on 23 June 2026. Pin exact versions in package.json, packageManager, Docker images, and CI configuration. Review these versions quarterly, but keep the architecture and project goal unchanged.

Category	Package / Tool	Required Version	Notes
Runtime	Node.js	24.16.0 LTS recommended; 22.x LTS minimum	Use Node 24 for new development; keep Node 22 as compatibility floor until dependencies require otherwise. Set packageManager in root package.json and enable Corepack. Use strict mode, NodeNext module resolution, noImplicitAny, exactOptionalPropertyTypes.
Package manager	pnpm	11.8.0	
Language	TypeScript	6.0.3	
Monorepo	Turborepo	2.9.18	Use cacheable tasks for build, lint, test, and typecheck.
API server	Fastify	5.8.5	Recommended API framework for apps/api; keeps Node API layer lightweight.
Validation	Zod	4.4.3	Use for request validation, config schemas, and typed LLM output validation.
Database	PostgreSQL	18.x stable	Use PostgreSQL 18 for production/dev containers; avoid PostgreSQL 19 beta for production.
ORM	Prisma CLI / @prisma/client	7.8.0	Keep prisma and @prisma/client on the same version.
Containers	Docker Engine	29.x	Required for local Postgres/Redis and reproducible test environments.
GitHub REST	@octokit/rest	22.0.1	Use only inside packages/github-client.
GitHub GraphQL	@octokit/graphql	9.x	GraphQL client; keep all GitHub API access behind the github-client package.
GitHub App auth	@octokit/app	16.1.2	Required for org-wide GitHub App authentication.
Rate limiting	Custom token-bucket limiter	Project-owned implementation	Prefer this over adding a stale dependency; Bottleneck 2.19.5 remains optional only.
Crypto	@noble/ciphers	2.2.0	AES-GCM for encrypted token storage in multi-user mode.
Dashboard framework	Next.js	16.2.9	Dashboard app baseline.
UI runtime	React	19.2.7	Used by Next.js dashboard and components.
Server state	@tanstack/react-query	5.101.0	Recommended for dashboard API caching, retries, and loading/error states.
Styling	Tailwind CSS	4.x	Use stable v4 line; pin exact package versions during implementation.
Charts	Recharts	latest stable at implementation lock	Used for Phase 13 analytics; pin after initial spike because chart APIs can shift.
Icons	lucide-react	1.21.0	Lightweight SVG icon set

Unit/integration tests	Vitest	4.1.9	for dashboard. Primary test runner for packages.
E2E tests	Playwright	1.61.0	Dashboard and PR-flow E2E tests.
Formatter	Prettier	3.8.4	Pin with --save-exact.
Linting	ESLint	10.5.0	Use flat config with typescript-eslint.
Local LLM runtime	Ollama	latest stable at implementation lock	Run only on localhost or private LAN; no hosted LLM dependency.
Local LLM models	qwen2.5-coder	3b and 7b	3b for classification; 7b for summaries and README drafts.
Local LLM models	deepseek-coder-v2	16b	Use for heavier generation if hardware supports it.
Scheduling queue	BullMQ + Redis	pin latest stable at implementation lock	Use for scheduled scans and longer jobs if dashboard mode needs durable background processing.
CLI scheduling	node-cron	pin latest stable at implementation lock	Acceptable for simple local-only scheduled scans.

Appendix E - Refinements and Improvements Applied

The following improvements were applied to make the plan more implementation-ready while preserving the stated project goal: audit, modernize, document, secure, and continuously improve GitHub repositories.

- Pinned the main runtime, tooling, framework, database, test, lint, and local LLM stack to explicit major/minor versions where reliable current version information is available.
- Replaced the older Node.js 20-first recommendation with Node.js 24 LTS as the recommended runtime and Node.js 22 LTS as the minimum supported runtime.
- Updated the TypeScript baseline from 5.4+ to 6.0.x strict mode to match the current toolchain expectation.
- Corrected the Next.js rule wording: the pages router should not be labeled as simply deprecated; mixed pages/app routing should be treated as a consistency or migration-review signal.
- Clarified that Bottleneck is optional and that a project-owned token-bucket limiter is safer for long-term maintainability.
- Added a version-pinning discipline: exact versions should be locked in package.json, pnpm-lock.yaml, Docker images, and CI before Phase 1 implementation begins.
- Added a validation layer recommendation with Zod so API contracts, configuration files, and structured LLM outputs can be runtime-checked instead of only TypeScript-checked.
- Added production readiness checkpoints: dependency locking, CI typecheck/lint/test gates, secret scanning before PR creation, migration rollback planning, and network-egress checks proving no external LLM API calls are made.

Recommended Root package.json Baseline

```
{
  "private": true,
  "packageManager": "pnpm@11.8.0",
  "engines": { "node": ">=22 <27", "pnpm": ">=11 <12" },
  "scripts": {
    "build": "turbo run build",
    "typecheck": "turbo run typecheck",
    "lint": "turbo run lint",
    "test": "turbo run test",
    "format": "prettier --write ."
  },
  "devDependencies": {
    "typescript": "6.0.3",
    "turbo": "2.9.18",
    "eslint": "10.5.0",
    "prettier": "3.8.4",
    "vitest": "4.1.9",
```

```
    "playwright": "1.61.0"  
  }  
}
```

Additional Acceptance Gates

- Every package must pass typecheck, lint, and unit tests in CI before any PR automation feature can create real repository branches.
- Every LLM output used for README, roadmap, PR description, or classification must be validated against a schema and stored with prompt, model, timestamp, and source facts.
- Any destructive action, including git history rewriting, dependency major upgrades, or deletion of generated files, must require explicit human approval and a dry-run preview.
- Security findings must distinguish current-branch exposure from git-history exposure; the remediation path must rotate/revoke credentials before history cleanup.
- Dashboard views must implement loading, empty, error, and stale-data states for every repository list, finding list, roadmap, and analytics chart.